

- A função `fread` pode ser usada para ler vários elementos de array de tamanho fixo, fornecendo um ponteiro para o array em que os elementos serão armazenados e indicando o número de elementos a serem lidos.
- A função `fread` retorna o número de itens que ela leu com sucesso. Se esse número for menor que o terceiro argumento na chamada da função, então ocorreu um erro de leitura.

■ Terminologia

arquivo 351
 arquivo de acesso aleatório 360
 arquivo de entrada-padrão 351
 arquivo de saída-padrão 351
 arquivo sequencial 351
 banco de dados 351
 binary digit (dígito binário) 350
 bit 350
 bloco de controle de arquivo (FCB) 352
 byte 350
 campo 350
 caracteres 350
 chave de registro 351
 conjunto de caracteres 350
 descritor de arquivo 352
 deslocamento 364
 deslocamento no arquivo 358
 erro-padrão 351
`fclose`, função 354
`feof`, função 354
`fgetc`, função 352
`fgets`, função 352
`FILE`, tipo 352
`fopen`, função 355
`fprintf`, função 352
`fputs`, função 352
`fread`, função 352
`fscanf`, função 352
`fseek`, função 362
`fwrite`, função 352
`getchar`, função 352
 hierarquia de dados 350
 letras 350
 marcador de fim de arquivo 351
 modelo de entrada/saída formatada 360
 modo de abertura de arquivo 353
`NULL` 353
`offset` 364
 ponteiro de posição do arquivo 358
`printf`, função 352
`putchar`, função 352
 registro 350
`SEEK_CUR` 365
`SEEK_END` 365
`SEEK_SET` 364
 símbolos especiais 350
 sistema de gerenciamento de banco de dados (SGBD) 351
 sistema de processamento de transação 360
`stderr` (dispositivo de erro-padrão) 352
`stdin` (dispositivo de entrada-padrão) 352
`stdout` (dispositivo de saída-padrão) 352
 stream 351
 tabela de arquivo aberto 352
 zeros e uns 350

■ Exercícios de autorrevisão

11.1 Preencha os espaços em cada uma das sentenças:

- Basicamente, todos os itens de dados processados por um computador são reduzidos a combinações de _____ e _____.
- O menor item de dados que um computador pode processar é chamado de _____.
- Um(a) _____ é um grupo de registros relacionados.
- Dígitos, letras e símbolos especiais são chamados de _____.
- Um grupo de arquivos relacionados é chamado de _____.
- A função _____ fecha um arquivo.
- A função _____ lê dados de um arquivo de maneira semelhante à que `scanf` lê de `stdin`.
- A função _____ lê um caractere de um arquivo especificado.
- A função _____ lê uma linha de um arquivo especificado.
- A função _____ abre um arquivo.
- A função _____ normalmente é usada quando se lê dados de um arquivo em aplicações de acesso aleatório.
- A função _____ reposiciona o ponteiro de posição do arquivo para um local específico no arquivo.

11.2 Responda quais das seguintes afirmações são *verdadeiras* e quais são *falsas*. Em caso de afirmações *falsas*, justifique sua resposta.

- a) A função `fscanf` não pode ser usada para ler dados da entrada-padrão.
- b) Você precisa usar `fopen` explicitamente para abrir os streams de entrada-padrão, saída-padrão e erro-padrão.
- c) Um programa precisa chamar explicitamente a função `fclose` para fechar um arquivo.
- d) Se o ponteiro de posição do arquivo apontar para um local em um arquivo sequencial diferente do início do arquivo, o arquivo precisa ser fechado e reaberto para leitura a partir do início do arquivo.
- e) A função `fprintf` pode gravar na saída-padrão.
- f) Os dados nos arquivos de acesso sequencial sempre são atualizados sem que se tenha de gravá-los sobre outros dados.
- g) Não é necessário pesquisar todos os registros em um arquivo de acesso aleatório para encontrar um registro específico.
- h) Os registros nos arquivos de acesso aleatório não possuem comprimento uniforme.
- i) A função `fseek` só pode fazer a busca a partir do início de um arquivo.

11.3 Escreva uma única instrução por meio da qual cada um dos itens a seguir possa ser obtido. Suponha que cada uma dessas sentenças se aplique ao mesmo programa.

- a) Escreva uma instrução que abra o arquivo `"oldmast.dat"` para leitura e atribua o ponteiro de arquivo retornado a `oFPtr`.
- b) Escreva uma instrução que abra o arquivo `"trans.dat"` para leitura e atribua o ponteiro de arquivo retornado a `tFPtr`.
- c) Escreva uma instrução que abra o arquivo `"newmast.dat"` para gravação (e criação) e atribua o ponteiro de arquivo retornado a `nFPtr`.

- d) Escreva uma instrução que leia um registro do arquivo `"oldmast.dat"`. O registro consiste no inteiro `accountNum`, na string `name` e no valor de ponto flutuante `currentBalance`.
- e) Escreva uma instrução que leia um registro do arquivo `"trans.dat"`. O registro consiste no inteiro `accountNum` e no valor de ponto flutuante `re-alAmount`.
- f) Escreva uma instrução que grave um registro no arquivo `"newmast.dat"`. O registro consiste no inteiro `accountNum`, na string `name` e no valor de ponto flutuante `currentBalance`.

11.4 Encontre o erro em cada um dos segmentos de programa a seguir e explique como corrigi-lo.

- a) O arquivo referenciado por `fPtr` (`"pagáveis.dat"`) não foi aberto.

```
printf( fPtr, "%d%s%d\n", account, company, amount );
```

- b) `open( "recebimento.dat", "r+" );`

- c) A instrução a seguir deveria ler um registro do arquivo `"pagáveis.dat"`. O ponteiro de arquivo `payPtr` refere-se a esse arquivo, e o ponteiro de arquivo `recPtr` refere-se ao arquivo `"recebimento.dat"`:

```
scanf( recPtr, "%d%s%d\n", &account, company, &amount );
```

- d) O arquivo `"ferramentas.dat"` deveria ser aberto para acrescentar dados ao arquivo sem descartar os dados atuais.

```
if ( ( tFPtr = fopen( "ferramentas.dat", "w" ) ) != NULL )
```

- e) O arquivo `"cursos.dat"` deveria ser aberto para acrescentar sem modificar o conteúdo atual do arquivo.

```
if ( ( cFPtr = fopen( "cursos.dat", "w+" ) ) != NULL )
```

■ Respostas dos exercícios de autorrevisão

11.1 a) 1s, 0s. b) Bit. c) Arquivo. d) Caracteres. e) Banco de dados. f) `fclose`. g) `fscanf`. h) `fgetc`. i) `fgets`. j) `fopen`. k) `fread`. l) `fseek`.

11.2 a) Falso. A função `fscanf` pode ser usada para ler da entrada-padrão com a inclusão do ponteiro para o stream de entrada-padrão, `stdin`, na chamada a `fscanf`.

b) Falso. Esses três streams são abertos automaticamente pela linguagem em C quando a execução do programa é iniciada.

c) Falso. Os arquivos serão fechados quando a execução do programa terminar, mas todos os arquivos deverão ser fechados explicitamente com `fclose`.

d) Falso. A função `rewind` pode ser usada para reposicionar o ponteiro de posição do arquivo para o início do arquivo.

e) Verdadeiro.

f) Falso. Na maioria dos casos, os registros de arquivo sequencial não possuem tamanho uniforme. Portanto, é

possível que a atualização de um registro faça com que outros dados sejam modificados.

g) Verdadeiro.

h) Falso. Os registros em um arquivo de acesso aleatório normalmente possuem tamanho uniforme.

i) Falso. É possível fazer buscas a partir do início do arquivo, a partir do final do arquivo e a partir do local atual no arquivo.

- 11.3**
- a) `ofPtr = fopen( "oldmast.dat", "r" );`
 - b) `tfPtr = fopen( "trans.dat", "r" );`
 - c) `nfPtr = fopen( "newmast.dat", "w" );`
 - d) `fscanf( ofPtr, "%d%s%f", &accountNum, name, ¤tBalance );`
 - e) `fscanf( tfPtr, "%d%f", &accountNum, &dollarAmount );`
 - f) `fprintf( nfPtr, "%d %s %.2f", accountNum, name, currentBalance );`

- 11.4**
- a) Erro: O arquivo "payables.dat" não foi aberto antes da referência ao seu ponteiro de arquivo.

Correção: Use `fopen` para abrir "payables.dat" para gravação, acréscimo ou atualização.

b) Erro: A função `open` não é uma função em C padrão.

Correção: Use a função `fopen`.

c) Erro: A função `fscanf` usa o ponteiro de arquivo incorreto para se referir ao arquivo "payables.dat".

Correção: Use o ponteiro de arquivo `payPtr` para se referir a "payables.dat".

d) Erro: O conteúdo do arquivo é descartado porque o arquivo é aberto para gravação ("w").

Correção: Para acrescentar dados ao arquivo, abra o arquivo para atualização ("r+"), ou abra-o para acréscimo ("a").

e) Erro: O arquivo "courses.dat" é aberto para atualização no modo "w+", que descarta o conteúdo atual do arquivo.

Correção: Abra o arquivo no modo "a".

Exercícios

- 11.5** Preencha os espaços em cada uma das sentenças:

- a) Computadores armazenam grandes quantidades de dados em dispositivos de armazenamento secundários, como _____.
- b) Um(a) _____ é composto(a) por vários campos.
- c) Um campo que pode conter dígitos, letras e espaços é chamado de campo _____.
- d) Para facilitar a recuperação de registros específicos de um arquivo, um campo em cada registro é escolhido como um(a) _____.
- e) A maioria das informações armazenadas nos sistemas de computadores é armazenada em arquivos _____.
- f) Um grupo de caracteres relacionados, que transmitem um significado, é chamado de _____.
- g) Os ponteiros de arquivo para os três arquivos que são abertos automaticamente quando a execução do programa é iniciada são chamados de _____, _____ e _____.
- h) A função _____ grava um caractere em um arquivo especificado.
- i) A função _____ grava uma linha em um arquivo especificado.
- j) A função _____ geralmente é usada para gravar dados em um arquivo de acesso aleatório.

- k) A função _____ reposiciona o ponteiro de posição do arquivo para o início do arquivo.

- 11.6** Indique quais das seguintes afirmações são *verdadeiras* e quais são *falsas*. Em caso de afirmações *falsas*, justifique sua resposta.

- a) As impressionantes funções executadas pelos computadores basicamente envolvem a manipulação dos números zeros e 1.
- b) As pessoas preferem manipular bits em vez de caracteres e campos, pois bits são mais compactos.
- c) As pessoas especificam programas e itens de dados como caracteres; os computadores, então, manipulam e processam esses caracteres como grupos de números zero e 1.
- d) O CEP do endereço de uma pessoa é um exemplo de campo numérico.
- e) O endereço de uma pessoa geralmente é considerado como um campo alfabético nas aplicações de computação.
- f) Os itens de dados processados por um computador formam uma hierarquia de dados em que os itens de dados se tornam maiores e mais complexos à medida que evoluímos de campos para caracteres para bits etc.

- g) Uma chave de registro identifica se um registro pertence a um campo em particular.
- h) A maioria das empresas armazena suas informações em um único arquivo para facilitar o processamento pelo computador.
- i) Os arquivos sempre são referenciados por nome nos programas em C.
- j) Quando um programa cria um arquivo, o arquivo é automaticamente retido pelo computador para futura referência.

11.7 Combinação de arquivos. O Exercício 11.3 pediu ao leitor que escrevesse uma série de instruções isoladas. Na realidade, essas instruções formam o núcleo de um importante tipo de programa de processamento de arquivo, a saber, um programa de união de arquivo. No processamento de dados comercial, é comum haver vários arquivos em cada sistema. Em um sistema de contas a receber, por exemplo, geralmente existe um arquivo mestre que contém informações detalhadas sobre cada cliente, como seu nome, endereço, número de telefone, saldo pendente, limite de crédito, termos de desconto, acordos contratuais e possivelmente um histórico resumido de compras recentes e pagamentos à vista.

À medida que as transações ocorrem (isto é, vendas são feitas e pagamentos são recebidos), elas são incluídas em um arquivo. Ao final de cada período de negócios (para algumas empresas, esse período é de um mês, para outras, uma semana, e ainda, em alguns casos, um dia), o arquivo de transações (chamado “trans.dat” no Exercício 11.3) é aplicado ao arquivo mestre (chamado “oldmast.dat” no Exercício 11.3), atualizando, assim, o registro de compras e pagamentos de cada conta. Após cada uma dessas rodadas de atualização, o arquivo mestre é regravado como um novo arquivo (“newmast.dat”), que é então usado no final do período seguinte, de negócios para dar início novamente ao processo de atualização.

Os programas de combinação de arquivo precisam lidar com certos problemas que não existem nos programas de arquivo único. Por exemplo, uma combinação nem sempre ocorre. Um cliente no arquivo mestre pode não ter feito compras ou pagamentos no período atual de negócios e, portanto, nenhum registro desse cliente aparecerá no arquivo de transação. De modo semelhante, um cliente que não fez compras ou pagamentos poderia ter mudado de endereço, e a empresa pode não ter tido a chance de criar um registro mestre para esse cliente.

Use as instruções escritas no Exercício 11.3 como base para um programa completo de contas a receber por combinação de arquivo. Use o número de conta em cada arquivo como chave de registro para fins de combinação. Suponha que cada arquivo seja um arquivo sequencial,

com registros armazenados em ordem crescente de número de conta.

Quando houver uma combinação (registros com o mesmo número de conta que aparecem no arquivo mestre e no arquivo de transação), some o valor no arquivo de transação ao saldo atual no arquivo mestre e grave o registro de “newmast.dat”. (Suponha que as compras sejam indicadas no arquivo de transação por valores positivos, e os pagamentos sejam indicados por valores negativos.) Quando houver um registro mestre para determinada conta, mas nenhum registro de transação correspondente, simplesmente grave o registro mestre em “newmast.dat”. Quando houver um registro de transação, mas nenhum registro mestre correspondente, imprima a mensagem “Registro de transação sem correspondência para a conta número...” (preencha o número de conta a partir do registro da transação).

11.8 Criação de dados para o programa de combinação de arquivos. Depois de escrever o programa do Exercício 11.7, escreva um programa simples que crie alguns dados de teste para verificar o programa do Exercício 11.7. Use os seguintes dados de conta como exemplo:

Arquivo mestre:		
Número de conta	Nome	Saldo
100	Alan Jones	348.17
300	Mary Smith	27.19
500	Sam Sharp	0.00
700	Suzy Green	-14.22

Arquivo de transação:	
Número de conta	Valor
100	27.14
300	62.11
400	100.56
900	82.17

11.9 Execute o programa do Exercício 11.7 usando os arquivos de dados de teste criados no Exercício 11.8. Use o programa de listagem do Exercício 11.7 para imprimir o novo arquivo mestre. Verifique os resultados com cuidado.

11.10 Combinação de arquivos com múltiplas transações. É possível (na realidade, é comum) ter vários registros de transação com a mesma chave de registro. Isso ocorre porque determinado cliente poderia fazer várias compras e pagamentos durante um período de negócios. Reescreva seu programa de combinação de arquivos para contas a receber do Exercício 11.7, a fim de prever a possibi-

lidade de tratar de vários registros de transação com a mesma chave de registro. Modifique os dados de teste do Exercício 11.8 para incluir os seguintes registros de transação adicionais:

Número de conta	Valor
300	83.89
700	80.78
700	1.53

11.11 Escreva instruções que executem cada uma das tarefas a seguir. Suponha que a estrutura

```
struct pessoa {
    char sobrenome[ 15 ];
    char nome[ 15 ];
    char idade[ 4 ];
};
```

tenha sido definida, e que o arquivo já esteja aberto para gravação.

- Inicializar o arquivo “nameage.dat” de modo que existam 100 registros com `lastName` = “unsigned”, `firstName` = “” e `age` = “0”.
- Inserir 10 sobrenomes, nomes e idades, gravando-os no arquivo.
- Atualizar um registro; se não houver informações no registro, dizer ao usuário “Nenhuma informação”.
- Excluir um registro que tenha informações, reiniciando esse registro em particular.

11.12 Inventário de material. Você é o dono de uma loja de materiais e precisa manter um inventário que poderá informar quais e quantas ferramentas você possui, e o custo de cada uma. Escreva um programa que inicialize o arquivo “hardware.dat” para 100 registros vazios, e permita que você insira os dados referentes a cada ferramenta, liste todas as suas ferramentas, exclua um registro para a ferramenta que não possui mais e atualize *qualquer* informação no arquivo. O número de identificação da ferramenta deverá ser o número do registro. Use a informação a seguir para iniciar seu arquivo:

# registro	Nome da ferramenta	Quantidade	Custo
3	Furadeira	7	57.98
17	Martelo	76	11.99
24	Serrote	21	11.00
39	Cortador de grama	3	79.50
56	Serra elétrica	18	99.99
68	Chave de fenda	106	6.99
77	Marreta	11	21.50
83	Chave inglesa	34	7.50

11.13 Gerador de texto para número de telefone. Os teclados-padrão de telefone contêm os dígitos de 0 a 9.

Os números de 2 a 9 possuem três letras associadas a cada um deles, conforme indicado pela tabela a seguir:

Dígito	Letra	Dígito	Letra
2	A B C	6	M N O
3	D E F	7	P Q R S
4	G H I	8	T U V
5	J K L	9	W X Y

Muitas pessoas acham difícil decorar números de telefone, por isso associam os dígitos a palavras de oito letras que correspondam aos números de telefone. Por exemplo, uma pessoa cujo número de telefone 3663-2272 poderia usar a tabela acima e associar seu telefone à palavra de sete letras “FONECASA”.

Constantemente, empresas tentam obter números de telefone que sejam facilmente lembrados por seus clientes. Então, se uma empresa puder anunciar uma palavra simples que seus clientes possam ‘digitar’, sem dúvida, a empresa receberá mais chamadas.

Cada palavra de oito letras corresponde a exatamente um número de telefone de oito dígitos. O restaurante que deseja aumentar seus negócios de pedido por telefone pode, certamente, fazer isso com o número 3686-3427(ou seja, “ENTREGAS”).

Cada número de telefone de oito dígitos corresponde a muitas palavras de oito letras separadas. Infelizmente, a maior parte delas representa justaposições irreconhecíveis de letras. É possível, porém, que o proprietário de um salão de cabeleireiro fique satisfeito em saber que o número de telefone 2662-6683 corresponde a “BOMCORTE”. O proprietário de uma loja de bebidas, sem dúvida, poderia ficar feliz sabendo que o número de telefone da loja, 2378-3527, corresponde a “CERVEJAS”.

Escreva um programa em C que, dado um número de oito dígitos, grave em um arquivo cada palavra de oito letras possível que corresponda a esse número. Existem 2187 (3 à sétima potência) dessas palavras. Evite números de telefone com os dígitos 0 e 1.

11.14 Modificação do gerador de texto para número de telefone. Se você tiver um dicionário computadorizado à disposição, modifique o programa que escreveu no Exercício 11.13 para pesquisar as palavras no dicionário. Algumas combinações de oito letras criadas por esse programa consistem em duas ou mais palavras (o número de telefone 3663-5467 produz “ÉOMELHOR”).

11.15 Modifique o exemplo da Figura 8.14 para usar as funções `fgetc` e `fputs` no lugar de `getchar` e `puts`. O programa deve dar ao usuário a opção de ler da entrada-padrão e gravar na saída-padrão ou ler de um arquivo especificado e gravar em um arquivo especificado. Se o usuário

escolher a segunda opção, peça a ele que digite os nomes do arquivo para os arquivos de entrada e saída.

11.16 Gravação dos tamanhos de tipo em um arquivo.

Escreva um programa que use o operador `sizeof` para determinar os tamanhos, em bytes, dos diversos tipos de dados no seu sistema de computação. Escreva os resultados no arquivo “`datasize.dat`” para que você possa imprimir os resultados mais tarde. O formato para os resultados no arquivo deverá ser o seguinte:

Tipo de dados	Tamanho
Char	1
unsigned char	1
short int	2
unsigned short int	2
Int	4
unsigned int	4
long int	4
unsigned long int	4
Float	4
double	8
long double	16

[Nota: os tamanhos de tipo no seu computador poderão ser diferentes daqueles que listamos aqui.]

11.17 Simpletron com processamento de arquivos.

No Exercício 7.19, você escreveu uma simulação de software de um computador que usava uma linguagem de máquina especial, chamada Simpletron Machine Language (SML). Na simulação, toda vez que você queria executar um programa SML, entrava com o programa no simulador por meio do teclado. Se você cometesse um erro durante a digitação do programa SML, o simulador era reiniciado e o código SML era reinserido. Seria bom ler o programa SML de um arquivo em vez de ter de digitá-lo várias vezes. Isso reduziria o tempo e os erros na preparação da execução dos programas SML.

- Modifique o simulador que você escreveu no Exercício 7.19 para que seja possível ler os programas SML a partir de um arquivo especificado pelo usuário no teclado.
- Após a execução do Simpletron, ele mostra o conteúdo de seus registradores e da memória na tela. Seria bom capturar a saída em um arquivo; portanto, modifique o simulador para que ele grave sua saída em um arquivo, além de exibi-la na tela.

Fazendo a diferença

- Verificador de phishing.** Phishing é uma forma de roubo de identidade em que, em um e-mail, um remetente se passa por uma fonte confiável para tentar adquirir informações particulares, como nomes de usuário, senhas, números de cartão de crédito e número de CPF. E-mails de phishing com mensagens que declaram ser de bancos conhecidos, empresas de cartão de crédito, sites de leilão, redes sociais e serviços de pagamento on-line podem parecer legítimos. Essas mensagens fraudulentas normalmente oferecem links para sites falsos, onde é solicitado que você digite informações confidenciais.

Visite a McAfee® (<www.mcafee.com/us/threat_center/anti_phishing/phishing_top10.html>), Security Extra (<www.securityextra.com/>), <www.snopes.com> e outros sites para encontrar listas dos principais e-mails de phishing. Também verifique o Anti-Phishing Working Group (<www.anti-phishing.org/>) e o site Cyber Investigations do FBI (<www.fbi.gov/cyberinvest/cyberhome.htm>),

onde você encontrará informações sobre as fraudes mais recentes e como se proteger delas.

Crie uma lista de 30 palavras, frases e nomes de empresa normalmente encontrados nas mensagens de phishing. Atribua um valor em pontos para cada um, com base na estimativa de probabilidade de estar em uma mensagem de phishing (por exemplo, um ponto se for pouco provável, dois pontos se for moderadamente provável ou três pontos se for altamente provável). Escreva um programa que varra um arquivo de texto para esses termos e frases. Para cada ocorrência de uma palavra-chave ou frase dentro do arquivo de texto, inclua o valor em pontos atribuído ao total de pontos para essa palavra ou frase. Para cada palavra-chave ou frase encontrada, exiba uma linha com a palavra ou frase, o número de ocorrências e o total de pontos. Depois, mostre o total de pontos da mensagem inteira. Seu programa atribui um total alto de pontos para alguns e-mails de phishing reais que você tenha recebido? Ele atribui um total alto de pontos para alguns e-mails legítimos que você tenha recebido?